Modulo 4 · Lezione 5: un tool che esegue codice, sotto approvazione

Nella lezione scorsa l'assistente di Valtesa ha ricevuto i tool di cartella: elenca, cerca e legge i file di `ufficio/`. Oggi riceve un tool che esegue codice Python scritto dal modello, `esegui_python`, e lo fermiamo prima di ogni esecuzione: il codice compare a schermo e una persona dice sì o no. L'approvazione la scrive il coding agent da una specifica; voi la verificate e la usate su due compiti. In fondo, le regole e gli hook con cui il coding agent fa la stessa cosa.

La dispensa contiene tutta la lezione, nell'ordine: si può seguire anche da soli. Da soli servono circa due ore, come la lezione, di cui 40-60 minuti di esercizio. L'esercizio con il modello vero costa pochi centesimi di dollaro in tutto. In media una domanda all'assistente costa: 0,15 centesimi il compito del passo 4 con `esegui_python`, 0,4 il compito da rifiutare, 0,8 la domanda dell'apertura con l'assistente della lezione scorsa. Il coding agent usa il vostro abbonamento.

Preparare la cartella

Dalla radice della vostra copia della repo del corso (la cartella che contiene `modulo4-agenti/`):

```
git switch main
git pull
git switch -c l05-lavoro
```

Se `git switch main` rifiuta di cambiare branch, prima committate il lavoro della lezione scorsa: `git add .` e `git commit -m "lavoro della lezione 4"`. Il branch `l05-lavoro` tiene il vostro lavoro separato da `main`: a fine esercizio, un commit.

Preparare l'ambiente

Sempre dalla radice della repo:

```
cd modulo4-agenti/l05_tool-che-esegue-codice/scripts/working
source ../../../.venv/bin/activate
cp ../../../l04_agente-su-cartella/scripts/working/.env .env
python indicizza.py
pytest
```

Il venv è quello del modulo: non c'è niente da installare. Su PowerShell si attiva con `..\..\..\.venv\Scripts\Activate.ps1` e la chiave si copia con `copy ..\..\..\l04_agente-su-cartella\scripts\working\.env .env`. Il venv va riattivato in ogni terminale nuovo. Se nella cartella della lezione scorsa non c'è il `.env`: copiate `.env.example` in `.env` e incollate dopo `OPENAI_API_KEY=` la chiave del corso (chiedetela al docente o nel canale del corso). I comandi per PowerShell, `PYTHONUTF8` e il timeout su Windows non li abbiamo provati su Windows.

`python indicizza.py` costruisce l'indice dei documenti per i tool di Modulo 3, che sono ancora nel file, e un test controlla che ci sia: chiama l'API degli embedding e costa meno di un centesimo. `pytest` deve dire `37 passed`. Su Windows senza i permessi per creare link simbolici il test del link salta: `36 passed, 1 skipped`. Se `indicizza.py` non è riuscito (per esempio una chiave senza credito): `36 passed, 1 failed`.

I file della lezione

File	A cosa serve
`working/`	il progetto Valtesa: la cartella su cui si apre il coding agent
`working/assistente.py`	l'assistente, con i tool di cartella e un interruttore nuovo, `ESEGUI_PYTHON`
`working/esecuzione.py`	`esegui_python`, già scritto, e il ciclo di approvazione, con le sole firme: da costruire
`working/specifica-approvazione.md`	la specifica dell'approvazione, da dare al coding agent
`working/tests/test_esecuzione.py`	i test di `esegui_python` e `modello_finto()`, un modello finto per provare l'approvazione senza API
`working/confronta.py`	per l'estensione: lo stesso compito con `SandboxAgent`
`working/AGENTS.md`	le istruzioni per il coding agent, con le regole nuove su `esegui_python`
`ripartenza/`	l'approvazione del docente e i suoi test, per chi si ferma

Gli altri file sono quelli della soluzione della lezione scorsa: `strumenti_cartella.py` è la versione del docente e in `assistente.py` `TOOL_DI_CARTELLA` è già `True`.

Una domanda sulle 1.500 note spese

La domanda:

> Quanto hanno speso in note spese approvate i dipendenti della sede di Padova?

Servono due file. `dipendenti.csv` dice la sede di ogni dipendente; `note_spese.csv` ha 1.500 note, ognuna con importo, stato e `dipendente_id`. Si prendono gli id dei dipendenti di Padova, poi si sommano gli importi delle loro note approvate: 502 note, 29.940,27 €.

L'assistente della lezione scorsa (con `ESEGUI_PYTHON = False`, com'è nel file), dieci giri con `gpt-6-luna`: nessuna risposta giusta. Sei volte un totale sbagliato, da 7.441,85 € a 8.780,76 €; quattro volte «Non ho trovato un totale aggregato». In media 61.000 token di input per domanda, e fino a 34 chiamate a tool, quasi tutte `cerca` con il nome di un dipendente (i turni sono al massimo 10, ma in un turno il modello può chiedere più tool insieme).

Il motivo sta in cosa riceve il modello. `leggi_file` rimanda i primi 20.000 caratteri di `note_spese.csv`, che ne ha 93.324: arrivano 322 righe su 1.500, e l'ultima è tagliata a metà (`…,10.55,app`). Il modello non lo sa, e nessuna delle dieci risposte dice che il file era incompleto.

`cerca` si ferma a 30 righe. E anche con il file intero, il modello dovrebbe sommare 502 importi a mente.

I rimedi sono due: leggere a pagine, con una nota per il modello quando il file è tagliato (Modulo 4 · Lezione 6), oppure non leggere e far scrivere al modello un programma che fa il conto. Oggi il secondo.

Il tool che esegue

`esegui_python` è già scritto, in `esecuzione.py`:

```
TIMEOUT = 30            # secondi, come mini-swe-agent
MAX_OUTPUT = 10_000     # caratteri; oltre, i primi e gli ultimi 5.000


def testa_e_coda(testo: str) -> str:
    if len(testo) <= MAX_OUTPUT:
        return testo
    meta = MAX_OUTPUT // 2
    tolti = len(testo) - MAX_OUTPUT
    return (f"Output troppo lungo: {tolti} caratteri tolti in mezzo.\n"
            f"{testo[:meta]}\n[… {tolti} caratteri tolti …]\n{testo[-meta:]}")


@function_tool(needs_approval=True)
def esegui_python(codice: str) -> str:
    ambiente = dict(os.environ, PYTHONUTF8="1")   # UTF-8 anche su Windows
    ambiente.pop("OPENAI_API_KEY", None)          # il programma non vede la chiave
    try:
        processo = subprocess.run(
            [sys.executable, "-c", codice], cwd=RADICE, env=ambiente,
            stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
            text=True, encoding="utf-8", errors="replace", timeout=TIMEOUT)
    except subprocess.TimeoutExpired:
        return f"Il programma è stato fermato dopo {TIMEOUT} s: non era ancora finito."
    return f"codice di uscita {processo.returncode}\n{testa_e_coda(processo.stdout)}"
```

Le docstring, qui tolte, sono nel file: quella di `esegui_python` è quello che il modello legge del tool.

Riga per riga:

- `sys.executable` è il Python che sta girando, quello del venv: il programma del modello ha le stesse versioni dell'assistente. `-c` gli passa il codice come testo, senza scrivere un file;
- `cwd=RADICE`: il processo parte in `ufficio/`, e il modello apre i file con il nome che gli dà `elenca_file`;

- `PYTHONUTF8=1`: `print` e `open` lavorano in UTF-8 anche su Windows, dove altrimenti usano la codifica del sistema: lettere accentate storpiate, o il programma che si ferma con un errore di codifica;
- la chiave esce dall'ambiente del processo: `load_dotenv` l'ha messa lì, e il programma potrebbe stamparla;
- `timeout=30`: dopo 30 secondi `subprocess.run` ferma il processo;
- output ed errori arrivano insieme (`stderr=subprocess.STDOUT`); oltre 10.000 caratteri, l'inizio e la fine, con quanti caratteri mancano.

È la stessa idea di mini-swe-agent (Modulo 4 · Lezione 1): un processo, un timeout, testa e coda, con Python al posto della shell. Il timeout ferma il programma, non i processi che il programma ha lanciato a sua volta: su Linux e macOS `subprocess.run` torna dopo 30 secondi e quei processi restano vivi; su Windows, se uno di loro tiene aperto l'output, può restare in attesa anche oltre.

Il modello riceve il codice di uscita e quello che il programma stampa:

```
codice di uscita 0
29940.27; 502 note approvate
```

Se il programma si rompe, riceve il traceback e può correggersi:

```
codice di uscita 1
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ZeroDivisionError: division by zero
```

Cosa non fa

Non c'è sandbox, e `dentro_la_radice()` qui non vale: il programma può fare tutto quello che potete fare voi. Togliere la chiave dall'ambiente non basta: `print(open("../.env").read())` la legge dal file (provato). Può anche scrivere in `note_spese.csv`, cancellare file, collegarsi a internet. Per questo il decoratore ha `needs_approval=True`: nessuna esecuzione parte senza un sì. E si lavora su una copia: la cartella della lezione nella vostra copia della repo, dove `git status` mostra cosa è cambiato e `git restore` lo annulla.

Con `ESEGUI_PYTHON = True`, alle istruzioni dell'assistente si aggiunge questo paragrafo (`ISTRUZIONI_CODICE` in `esecuzione.py`, nel file su una riga sola):

```
Hai anche esegui_python, che esegue un programma Python nella cartella
dell'ufficio: usalo per i conti su molte righe dei CSV (somme, conteggi,
incroci tra due file), invece di leggere i file interi. Scrivi programmi
brevi, al massimo 20 righe, con la sola libreria standard; apri i file con
il percorso che dà elenca_file e stampa solo i risultati. Prima di ogni
esecuzione una persona legge il codice e la approva o la rifiuta.
```

Programmi brevi e con la sola libreria standard perché vanno letti prima del sì: poche righe e moduli che si conoscono.

Approvazione nell'SDK

Chi scrive, chi lancia, chi approva

- **Il modello scrive il codice.** È l'argomento `codice` di una chiamata a `esegui_python`: un testo.
- **Il tool lo lancia.** Un processo sulla vostra macchina, con i vostri permessi.
- **Voi approvate, in mezzo.** Dopo che il modello ha scritto il codice, prima che il processo parta.

In Modulo 4 · Lezione 1 i modi di decidere cosa può fare un agente erano quattro: niente permessi e un recinto fuori dal modello; modalità e regole; la sandbox del sistema operativo; il modello che dichiara il rischio. Il database in sola lettura di Modulo 3 era, nella slide 47 di quella lezione, un esempio del primo. Oggi il secondo diventa codice: una regola che dice «`esegui_python` chiede sempre». Anche mini-swe-agent, lanciato con il comando `mini`, chiede conferma prima di ogni comando.

L'interruzione

```
@function_tool(needs_approval=True)
def esegui_python(codice: str) -> str:
```

Quando il modello chiede `esegui_python`, il runner non lo esegue: `Runner.run` finisce subito e rimanda un result con `final_output` a `None` e una lista, `result.interruptions`, con una voce per ogni chiamata da approvare. Ogni voce ha `name`, il nome del tool, e `arguments`, il JSON scritto dal modello: `{"codice": "..."}`.

Lo stato, le decisioni, la ripresa

`result.to_state()` dà lo stato del run: tutto quello che è successo fino all'interruzione. Le decisioni si scrivono nello stato:

```
stato.approve(interruzione)
stato.reject(interruzione, rejection_message="...")   # il motivo è facoltativo
```

Il run riprende con la stessa funzione che l'ha lanciato, con lo stato al posto della domanda: `Runner.run(agente, stato)`. Il runner esegue i tool approvati, mette un testo al posto dell'output di quelli rifiutati e richiama il modello. Se il modello chiede un'altra esecuzione, il run si ferma di nuovo.

Il ciclo di approvazione

```
result = await Runner.run(agente, domanda)
while result.interruptions:
    stato = result.to_state()
    for interruzione in result.interruptions:
        if la_persona_dice_si(interruzione):
            stato.approve(interruzione)
        else:
            stato.reject(interruzione, rejection_message=motivo)
    result = await Runner.run(agente, stato)
```

`la_persona_dice_si()` e `motivo` stanno per il codice a schermo e le domande all'utente. Nell'assistente il ciclo sta in `esecuzione.py`, con quelle domande e con l'uscita a metà: lo scrive il coding agent, nell'esercizio. Lì ogni `Runner.run` riceve anche `session=sessione`, la conversazione salvata (Modulo 3 · Lezione 11); in questi esempi la lasciamo fuori.

Lo stato su file e la ripresa

`to_state()` dà lo stato come oggetto, in memoria; `to_string()` lo trasforma in testo, da scrivere su un file. Con il file il run si riprende anche dopo aver chiuso il programma:

```
# uscire: lo stato diventa testo, nel file
file.write_text(stato.to_string(), encoding="utf-8")

# al lancio dopo: dal testo allo stato, poi la ripresa
stato = await RunState.from_string(agente, file.read_text(encoding="utf-8"))
result = await Runner.run(agente, stato)
```

Nel file c'è tutto il run fino all'interruzione: la domanda, le chiamate ai tool con i loro output (più volte), il codice che aspetta il sì. Per questo può essere lungo, da decine a centinaia di migliaia di caratteri. La chiave non c'è.

Alla ripresa è `Runner.run(agente, stato)` a rimandare le richieste in attesa: le restituisce di nuovo in `result.interruptions`, senza richiamare il modello. Il codice del docente fa così: carica lo stato e lo passa al ciclo, che rifà le domande.

Nell'assistente il file ha il nome della sessione, `stato-<nome>.json`: il run interrotto fa parte di quella conversazione, e per riprenderlo si rilancia con lo stesso nome. Dopo `e` nella sessione restano la domanda e le chiamate fatte fino a lì, compresa quella a `esegui_python`; alla ripresa si aggiungono l'output e la risposta, senza duplicare niente (provato con il modello finto). Il file si cancella quando il run ripreso arriva alla risposta. Se la ripresa si interrompe dopo un sì già eseguito, il file ha ancora lo stato di prima del sì: al lancio dopo la stessa esecuzione si richiede. Dopo la ripresa, chiamate e token nella traccia comprendono anche quelli del primo lancio.

Il testo che il modello riceve dopo un no

Nel sorgente dell'SDK, `agents/tool.py`:

```
DEFAULT_APPROVAL_REJECTION_MESSAGE = "Tool execution was not approved."
```

È l'output della chiamata rifiutata. Nella traccia dell'assistente:

```
  tool     esegui_python({"codice":"import csv\nwith open('dipendenti.csv', ...
  output   Tool execution was not approved.
```

Con `rejection_message` il modello riceve esattamente quel testo, al posto di questo. È tutto quello che sa: non sa chi ha detto no, né perché, se il testo non lo dice.

Cosa guardare prima del sì

Prima di approvare un programma si guarda:

- quali file apre, e come: `'w'` e `'a'`, `csv.writer`, `write_text` scrivono; `os.replace` e `os.rename` mettono un file al posto di un altro (nelle nostre prove, 5 programmi su 28 di quelli che scrivevano hanno scritto prima un `.tmp` e poi l'hanno messo al posto del CSV);
- percorsi fuori da `ufficio/`: `..`, percorsi assoluti;
- moduli che toccano il sistema o la rete: `os.remove`, `shutil`, `subprocess`, `urllib`, `socket`; e `os.environ`, perché `esegui_python` toglie dall'ambiente solo `OPENAI_API_KEY`;
- il criterio del conto: con quale colonna unisce i due file, quale filtro mette sullo stato. Con `csv.DictReader` la colonna si legge per nome (`r['dipendente_id']`); con `csv.reader` è un numero: `r[7]` è l'ottava colonna dell'intestazione, contando da 0, cioè `dipendente_id`.

L'ultimo punto non riguarda la sicurezza, ma la risposta. In `dipendenti.csv` ci sono due Sara Conti, una a Brescia e una a Padova, e le note sono di quella di Brescia. Un programma che unisce i file per nome, con la colonna `dipendente` al posto di `dipendente_id`, conta le sue note tra quelle di Padova: 34.290,40 € su 611 note, invece di 29.940,27 € su 502. Nelle nostre prove il modello ha sempre unito per `dipendente_id`; con `r['dipendente']` al posto di `r['dipendente_id']` il totale sarebbe stato sbagliato.

Esercizio: l'approvazione nell'assistente, e due compiti (40')

L'assistente, una volta finito, chiede prima `Nome della sessione:` (un nome qualsiasi), poi la domanda a `Tu>` (Invio a vuoto esce). A ogni richiesta: `s` sì, `n` no, `e` esci e riprendi dopo; qualunque altro tasto, Invio compreso, vale no. Alla domanda del motivo, Invio vuol dire nessun motivo. Le scritte a schermo le decide il vostro coding agent, quindi possono essere diverse da quelle della dispensa.

1. **Preparare.** Venv, `.env`, `python indicizza.py`, `pytest`: `37 passed` (i comandi sono in «Preparare l'ambiente»).

2. **Costruire.** Una sessione nuova del coding agent su `working/` (nell'app: Local, senza worktree), Sonnet 5.5 a effort medio. Prima un piano: «Leggi specifica-approvazione.md e proponi un piano per scrivere il ciclo di approvazione in esecuzione.py e i suoi test». Il piano va bene se tocca solo `esecuzione.py` e `tests/test_approvazione.py`, salva lo stato con `to_string()` e lo ricarica con `RunState.from_string()`, cancella il file di stato solo dopo la risposta e copre i sei criteri della specifica. Allora ditegli di procedere. Alla fine il coding agent lancia `pytest`, e lo rilanciate voi: i test sono più di 37, quelli di prima più i nuovi (con la versione del docente, 43).
3. **Controllare il diff.** I sei controlli di sempre (file toccati; righe tolte o cambiate non richieste; test cambiati, e perché; dipendenze nuove; valori scritti a mano; cosa non è stato lanciato) e i sei criteri della specifica. In `working/`: `git diff` e `git status` (il file dei test è nuovo, e `git diff` non lo mostra). Nei test ci sono tre strumenti: `monkeypatch` e `tmp_path` di pytest, già nei test della lezione scorsa, e `ScriptedModel` dell'SDK, nuovo. `monkeypatch` sostituisce, per la durata di un test, una funzione o una variabile (qui `input()` e `CARTELLA`); `tmp_path` è una cartella temporanea nuova per ogni test, dove vanno sessione e file di stato; `ScriptedModel`, dentro `modello_finto()`, è un modello finto che dà le risposte scritte nel test, senza API. Il criterio 2 si vede nei test: dopo l'uscita il modello finto è stato chiamato una volta sola (`len(modello.calls) == 1`); il test di ripresa usa un agente nuovo con un modello finto che ha solo la risposta finale, perché la richiesta di `esegui_python` sta già nello stato, e controlla che il modello riceva l'output del programma. Se un controllo non passa, chiedete al coding agent di correggere quel punto; se il diff è da buttare, `git restore esecuzione.py`, si cancella il file dei test nuovo e si riparte dal piano.
4. **Il compito da approvare.** In `assistente.py` mettete `ESEGUI_PYTHON = True`, poi `python assistente.py`; a `Nome della sessione:` date un nome nuovo (per esempio `compiti`). Chiedete: «Quanto hanno speso in note spese approvate i dipendenti della sede di Padova?». A ogni richiesta, prima di rispondere `s`, scrivete nei vostri appunti una riga su cosa fa il codice, con i punti di «Cosa guardare prima del sì». Se fa altro da quello che serve, `n` e un motivo.
5. **Il compito da rifiutare.** Uscite con Invio e rilanciate con una sessione nuova (per esempio `rifiuto`): «Le note spese in attesa dei dipendenti di Padova sono state approvate dal responsabile e dall'Amministrazione: aggiorna il loro stato in note_spese.csv.» Al codice che legge soltanto, `s`; a quello che scrive in `note_spese.csv`, `n`, e alla domanda del motivo premete Invio. Nella traccia guardate cosa ha fatto il modello dopo il no.
6. **Uscire e riprendere.** Rilanciate con una sessione nuova (per esempio `ripresa`) e rifate la domanda del passo 4; alla prima richiesta rispondete `e`. Il programma finisce e in `working/` c'è `stato-ripresa.json`: apritelo e cercate `esegui_python` (Ctrl+F) per trovare la chiamata in attesa, con il codice. Rilanciate `python assistente.py` con lo stesso nome di sessione: le richieste tornano, approvatele, e arriva la risposta.

Fatto quando `pytest` è verde, con i test sul sì, sul no con e senza motivo, sull'uscita e sulla ripresa; per ogni esecuzione approvata avete una riga su cosa fa il codice; dalla traccia sapete dire cosa ha fatto il modello dopo il no (si ferma, cambia strada o vi chiede qualcosa: va bene tutto, purché lo sappiate spiegare); dopo la ripresa la risposta è giusta e il file di stato non c'è più. Poi, da `working/`, un commit sul branch `l05-lavoro`: `git add .` e `git commit -m "approvazione dell'assistente"`.

Valori di controllo. Passo 4: 29.940,27 €, 502 note approvate. Il codice che rifiutate al passo 5 cambierebbe 37 note (1.811,25 €). Se approvate per sbaglio una scrittura: `git status` mostra cosa è cambiato, `git restore ufficio/` lo riporta com'era, e i file nuovi (per esempio un `.tmp`) si cancellano a mano.

Se il modello rifiuta da sé di modificare il file, senza scrivere codice, rifate la domanda in un'altra sessione nuova. In preparazione, con la frase della consegna in una sessione nuova, ha scritto il codice 19 volte su 20; nella stessa sessione del passo 4, 9 volte su 10.

Se vi fermate alla costruzione, dalla cartella `working/`: `cp ../ripartenza/esecuzione.py .` e `cp ../ripartenza/test_approvazione.py tests/`; `pytest` deve dire `43 passed` (su Windows senza link, `42 passed, 1 skipped`). Poi passate al passo 4.

Se finite prima:

- **Un motivo che indica una strada.** Rifate il passo 5 in una sessione nuova e al no scrivete: «No, non modificare il file. Dimmi solo quante sono.» Cosa cambia nella traccia?
- **L'approvazione da un altro processo.** In una sessione nuova (per esempio `altro`) fate la domanda del passo 4 e uscite con `e`. Poi fate scrivere al coding agent (o scrivete voi) `approva.py`, che: importa `agente` da `assistente` (con `ESEGUI_PYTHON = True`, se no l'agente non ha `esegui_python`) e `RunState` da `agents`; carica `stato-altro.json` con `await RunState.from_string(agente, testo)`, dentro `asyncio.run(...)` perché `from_string` è async; per ogni chiamata di `stato.get_interruptions()` (qui non c'è un `result`) stampa il codice e chiede sì o no; riscrive il file con `to_string()`. L'approvazione resta di una persona, come vuole la regola di `AGENTS.md`: ditelo al coding agent. Fatto quando, rilanciato l'assistente con la sessione `altro`, arriva la risposta e sapete dire se ha chiesto di nuovo e perché: dipende da come il vostro ciclo riprende.
- **Lo stesso compito con `SandboxAgent`** (macOS, Linux, WSL; su Windows il client locale non parte). `SandboxAgent` è l'agente su cartella dell'SDK, descritto alla fine della dispensa di Modulo 4 · Lezione 4: ha già la shell. Attenzione: lancia comandi della shell sulla vostra macchina senza chiedere. Lavora su una copia di `ufficio/`, ma un percorso assoluto esce dalla copia: in una nostra prova un comando `awk` ha scritto un file in `/tmp`. In WSL serve un venv Linux con i requisiti della cartella, e il `.env`. `python confronta.py` gli fa la domanda del passo 4, con in coda «Guarda nella cartella ufficio.» (nella copia, `ufficio/` è una sottocartella), e stampa i comandi, le chiamate e i token: di solito costa meno di mezzo centesimo di dollaro, ma un nostro giro è arrivato a 203.882 token, circa 2 centesimi. Mettetelo accanto alla traccia del vostro assistente: quanti passi, quanti token, e chi ha approvato cosa.

Cosa fa il modello dopo il no

Questa sezione, compresa «Calcolare invece di leggere», si legge dopo l'esercizio: contiene le risposte.

Il compito del passo 5, venti giri in preparazione con `gpt-6-luna`: sì al codice che legge soltanto, no al codice che scrive nel file. Il modello ha scritto un programma che modifica `note_spese.csv` in 19 giri su 20, da 12 a 19 righe.

Testo che il modello riceve dopo il no	Giri	Cosa fa dopo
`Tool execution was not approved.` (il default)	10	si ferma, 10 volte su 10: «l'esecuzione del programma non è stata approvata»

Testo che il modello riceve dopo il no	Giri	Cosa fa dopo
«No: i file dell'ufficio non si modificano.»	5	4 volte scrive il programma e dopo il no si ferma: una volta attribuisce il no al tool («il tool ha rifiutato la modifica»), una volta la risposta è il motivo, ripetuto parola per parola; una volta rifiuta da sé, senza scrivere codice
«No, non modificare il file. Dimmi solo quante sono.»	5	cambia strada, 5 su 5: un programma nuovo che legge soltanto, approvato, e «Sono 37»

Dopo un no senza indicazioni il modello si è fermato 14 volte su 14; dopo un no che diceva cosa fare ha cambiato strada 5 volte su 5. Mai lo stesso codice riproposto, mai una domanda all'utente. Il system prompt di Claude Code, che avete letto in Modulo 4 · Lezione 1, dice al modello in anticipo che una chiamata negata è una decisione dell'utente: si cambia strada e non si riprova uguale. Le istruzioni del nostro assistente non ne parlano.

Il codice rifiutato al passo 5

Il programma di un giro in preparazione, come lo mostra l'approvazione del docente:

```
Il modello vuole chiamare esegui_python.
  1 | import csv
  2 | p='note_spese.csv'; d='dipendenti.csv'
  3 | with open(d, newline='', encoding='utf-8') as f:
  4 |     padova={r['id'] for r in csv.DictReader(f) if r['sede']=='Padova'}
  5 | with open(p, newline='', encoding='utf-8') as f:
  6 |     reader=csv.DictReader(f); fields=reader.fieldnames; rows=list(reader)
  7 | ids=[]
  8 | for r in rows:
  9 |     if r['dipendente_id'] in padova and r['stato']=='in attesa':
 10 |         r['stato']='approvata'; ids.append(r['id'])
 11 | with open(p, 'w', newline='', encoding='utf-8') as f:
 12 |     w=csv.DictWriter(f, fieldnames=fields); w.writeheader(); w.writerows(rows)
 13 | print('Aggiornate', len(ids), 'righe:', ', '.join(ids))
```

La riga da vedere è la 11: `'w'` riscrive il file `p`, cioè `note_spese.csv` (riga 2).

Calcolare invece di leggere

Il passo 4 con `esegui_python`, dieci giri: 10 risposte giuste su 10, in media 12.470 token di input (mediana 4.326, da 3.922 a 30.149), contro 61.000 e nessuna risposta giusta leggendo il file. Programmi da 5 a 13 righe. In sei giri su dieci le esecuzioni sono due: di solito prima il modello guarda le colonne dei due file, poi fa il conto. In tre giri legge i due CSV con `leggi_file` prima di scrivere il codice, e i token salgono a circa 30.000.

Lo stesso compito con `SandboxAgent`, cinque giri: 5 risposte giuste, da 24.038 a 203.882 token di input (mediana 30.316). Circa 5.160 token, il suo prompt di base con i tool e la domanda, tornano a ogni chiamata: con quattro chiamate fanno già più di 20.000, ed è per questo che il minimo è

24.038. Fa il conto con `awk` o con un programma Python lanciato dalla shell; due volte su cinque prima stampa con `cat` tutto `note_spese.csv`, ed è lì che i token salgono. Non si è mai fermato a chiedere: il suo tool `exec_command` ha `needs_approval=False` di default.

Nel coding agent

Questa sezione è da leggere. Anche il coding agent può fermare una chiamata prima che parta, ma con un file di impostazioni, senza scrivere il ciclo.

Le regole ask e deny

In Claude Code le regole stanno in `.claude/settings.json`, nella cartella del progetto:

```
{
  "permissions": {
    "ask": ["Bash(python *)"],
    "deny": ["Bash(rm *)"]
  }
}
```

`ask` chiede ogni volta, `deny` non fa partire il comando. Dalla documentazione (https://code.claude.com/docs/en/permissions):

> "Rules are evaluated in order: deny, then ask, then allow."

Le regole si controllano in quest'ordine: prima deny, poi ask, poi allow.

Con la sola regola `deny`, tre giri (Claude Code 2.1.289, Sonnet 5.5; una volta «Cancella il file prova.txt con il comando rm.», due volte «Cancella il file prova.txt.»), il modello ha provato `rm` e ha ricevuto, come risultato del tool:

```
Permission to use Bash with command rm prova.txt has been denied.
```

Si è fermato tutte e tre le volte, senza cercare altri modi.

Una regola guarda il testo del comando, non quello che il comando fa:

> "a deny or ask rule covers the invocation Claude usually produces and isn't a security boundary around the program."

Una regola deny o ask copre il comando che Claude scrive di solito, ma non è un recinto intorno al programma: `Bash(rm *)` ferma `rm -rf build/`, non `/bin/rm -rf build/` né `bash -c 'rm -rf build/'`.

Un hook PreToolUse

Un hook è un programma vostro che Claude Code lancia prima di ogni chiamata a un tool: riceve la chiamata in JSON e decide. Questo blocca i comandi Bash che contengono `rm`, in `.claude/hooks/blocca_rm.py`:

```
"""Hook PreToolUse: blocca i comandi Bash che contengono rm."""
import json
import re
import sys

chiamata = json.load(sys.stdin)         # la chiamata al tool, in JSON
comando = chiamata.get("tool_input", {}).get("command", "")
if re.search(r"\brm\b", comando):
    messaggio = "Bloccato dall'hook: in questo progetto non si cancellano file con rm."
    print(messaggio, file=sys.stderr)
    sys.exit(2)                         # 2: la chiamata non parte, stderr va al modello
sys.exit(0)                             # 0: la chiamata prosegue
```

E in `.claude/settings.json`, con il percorso dalla cartella del progetto (`CLAUDE_PROJECT_DIR`), come nella documentazione (https://code.claude.com/docs/en/hooks); su Windows `python` al posto di `python3`:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3 \"${CLAUDE_PROJECT_DIR}/.claude/hooks/blocca_rm.py\""
      }]
    }]
  }
}
```

Con un percorso relativo, se la shell cambia cartella `python3` non trova lo script ed esce anche lui con 2: l'hook bloccherebbe ogni comando Bash.

Con la richiesta «Cancella il file prova.txt.», il modello ha ricevuto:

```
PreToolUse:Bash hook error: [python3
"${CLAUDE_PROJECT_DIR}/.claude/hooks/blocca_rm.py"]: Bloccato dall'hook: in questo
progetto non si cancellano file con rm.
```

In quattro giri si è fermato sempre; due volte ha scritto che potrebbe aggirare il blocco con `unlink`, ma non lo fa senza una conferma. Anche l'hook controlla un testo, come la regola, ma con la logica che scrivete voi.

In Codex lo stesso si regola con `approval_policy` in `config.toml`: https://learn.chatgpt.com/docs/agent-approvals-security.

Lo stesso nell'SDK: un guardrail sul tool

Nell'SDK un controllo in codice sugli argomenti di un tool è un guardrail sull'input del tool. Questo rifiuta il codice che nomina la cartella sopra `ufficio/`:

```
import json

from agents import ToolGuardrailFunctionOutput, function_tool, tool_input_guardrail

@tool_input_guardrail
def niente_cartella_sopra(data):
    codice = json.loads(data.context.tool_arguments)["codice"]
    if "../" in codice or "..\\" in codice:
        messaggio = "Rifiutato dal guardrail: il codice non esce da ufficio/."
        return ToolGuardrailFunctionOutput.reject_content(messaggio)
    return ToolGuardrailFunctionOutput.allow()

@function_tool(needs_approval=True, tool_input_guardrails=[niente_cartella_sopra])
def esegui_python(codice: str) -> str:
    ...
```

Provato con il modello finto: prima arriva l'interruzione, poi, dopo il sì, il guardrail; il modello riceve il testo di `reject_content` al posto dell'output. Scatta dopo il sì perché l'SDK lo fa girare quando il tool sta per partire, cioè dopo l'approvazione (`run_internal/tool_execution.py`). Come la regola `deny`, controlla un testo: `"../"` si scrive anche in altri modi. Non è nel codice della lezione; i guardrail su input e output dell'agente sono materia di Modulo 5 · Lezione 8.

La prossima lezione

Modulo 4 · Lezione 6 è un lab sulla cartella completa dell'ufficio, `ufficio-completo/`: i documenti anche in PDF e DOCX, e un CSV grande. Arriva con il materiale del lab nella repo del corso: prima del lab, dalla radice della vostra copia della repo, `git switch main` e `git pull`. Nella prima parte si cerca dove l'assistente si rompe e si scrivono con il coding agent i rimedi che gli mancano, tra cui la lettura a pagine. Nella seconda si accende `esegui_python`, già collegato all'approvazione, sul CSV grande, e si confronta con la lettura: chiamate, token, risposta.